

# CSCD602:

# Advanced Software Engineering

Session 4: Program Evolution Dynamics

By  
Solomon Mensah (PhD)



UNIVERSITY OF GHANA

# Agenda

- Introduction
- Laws of Lehman
- Implications of Lehman's Laws
- Case studies for Lehman's Laws
- Modern Software and Lehman's Laws
- Strategies for Managing Software Evolution

# Introduction

- **Program Evolution Dynamics**
  - study of the processes of system change.
  - study of how software systems evolve over time in response to changing requirements, environments, and constraints.
- **Key Aspects:**
  - Continuous change
  - Increasing complexity
  - Maintenance efforts
- Understanding these dynamics helps in predicting challenges and improving software longevity.

# Lehman's Laws Overview

- Lehman and his collaborators from IBM are generally credited with pioneering the research field of software evolution.
- They formulated a set of observations that he called laws of evolution.
- These laws are the results of studies of the evolution of large-scale proprietary or closed source system (CSS).

# Lehman's Laws Overview

- After major empirical studies of large software systems,
  - Lehman and Belady proposed that there were a number of 'laws' which applies to all systems as they evolved.
  - Provide insights into how and why software systems evolve.
  - They are applicable to large systems developed by large organisations.
  - Perhaps less applicable in other cases.



# Laws of Lehman

- *Continuing change* (1<sup>st</sup>) – A system will become progressively less satisfying to its user over time, unless it is continually adapted to meet new needs.
- *Increasing complexity* (2<sup>nd</sup>) – A system will become progressively more complex, unless work is done to explicitly reduce the complexity.
- *Self-regulation* (3<sup>rd</sup>) – Over time any measurements of the system or its process will follow a well-defined trend, with ripples in either direction that follow a normal distribution.
  - This law implies that software evolution is predictable, allowing project managers to estimate timelines and resource requirements by analyzing previous releases.

# Laws of Lehman

- *Conservation of organizational stability* (4<sup>th</sup>) – The average effective global activity rate on an evolving system does not change over time, that is the average amount of work that goes into each release is about the same.
  - The average rate of a program's development is constant and independent of the amount of work put into it.
  - Over a software system's lifetime, its overall rate of development remains relatively constant and is independent of the resources assigned to its design and development.

# Laws of Lehman

- *Conservation of familiarity* (5<sup>th</sup>) – The amount of new content in each successive release of a system tends to stay constant.
  - The average amount of new content in each release of a system remains constant.
- *Continuing growth* (6<sup>th</sup>) – The amount of functionality in a system will increase over time, in order to please its users.
  - To meet user needs, the amount of functional content in a system must increase over time.



# Laws of Lehman

- *Declining quality* (7<sup>th</sup>) – A system will be perceived as losing quality over time, unless its design is carefully maintained and adapted to new operational constraints.
- *Feedback system* (8<sup>th</sup>) – Complex feedback systems must be recognized as such to improve a product significantly.
  - Successfully evolving a software system requires recognition that the development process is a multi-loop, multi-agent, multi-level feedback system.
  - Evolution is a feedback-driven process.

# Lehman's Laws

| Law                         | Description                                                                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Continuing change           | A program that is used in a real-world environment necessarily must change or become progressively less useful in that environment.                                                         |
| Increasing complexity       | As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.                                    |
| Large program evolution     | Program evolution is a self-regulating process. System attributes such as size, time between releases and the number of reported errors is approximately invariant for each system release. |
| Organisational stability    | Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.                                                |
| Conservation of familiarity | Over the lifetime of a system, the incremental change in each release is approximately constant.                                                                                            |
| Continuing growth           | The functionality offered by systems has to continually increase to maintain user satisfaction.                                                                                             |
| Declining quality           | The quality of systems will appear to be declining unless they are adapted to changes in their operational environment.                                                                     |
| Feedback system             | Evolution processes incorporate multi-agent, multi-loop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.                         |



# Implications of Lehman's Laws

- **For Developers:**
  - Need for proactive maintenance strategies.
  - Manage increasing complexity.
- **For Project Managers:**
  - Plan for ongoing updates and resource allocation.
  - Address quality degradation over time.
- **For Organizations:**
  - Importance of long-term evolution planning.
  - Integration of feedback mechanisms.

# Case Studies

## **1. Linux Kernel Evolution**

- Demonstrates self-regulation and conservation of familiarity.

## **2. Enterprise Software**

- Highlights increasing complexity and declining quality.

# Modern Software and Lehman's Laws

- **Relevance Today:**
  - Applicable to both legacy and modern software systems.
  - Addresses challenges of continuous delivery and agile development.
- **New Considerations:**
  - Cloud-based and distributed systems.
  - Impact of AI and machine learning systems.

# Strategies for Managing Software Evolution

- **Complexity Management:**
  - Refactoring
  - Modular design
- **Quality Assurance:**
  - Regular testing
  - User feedback integration
- **Resource Planning:**
  - Dedicated maintenance teams
  - Continuous integration/continuous deployment (CI/CD) pipelines

# Summary

- Program evolution dynamics reveal patterns in software change.
- Lehman's Laws offer a framework for understanding software behavior over time.
- Adapting to these dynamics is crucial for sustainable software development.

# Thank you

